Review of Class 17: Search, Trees, and Traversals

Operator combinations, unique printing, Task B, expression trees, pre-/in-/post-order

CSCI 235 – Data Structures

Hunter College, Fall 2026

Review of Class 17 (Tuesday, November 3, 2026)

Roadmap: One Idea in Four Steps

Class 17 was one idea that kept getting a picture added.

1. **Grow an expression** by recursion: choose an operator, explore, undo.

Roadmap: One Idea in Four Steps

Class 17 was one idea that kept getting a picture added.

1. **Grow an expression** by recursion: choose an operator, explore, undo.
2. **Do not repeat yourself:** skip a letter this seat has already tried (`printUnique`); Make 24 Task B returns *one expression*.

Roadmap: One Idea in Four Steps

Class 17 was one idea that kept getting a picture added.

1. **Grow an expression** by recursion: choose an operator, explore, undo.
2. **Do not repeat yourself:** skip a letter this seat has already tried (`printUnique`); Make 24 Task B returns *one expression*.
3. **An expression is a tree.** The root is the *outermost* operation, done after its operands are evaluated.

Roadmap: One Idea in Four Steps

Class 17 was one idea that kept getting a picture added.

1. **Grow an expression** by recursion: choose an operator, explore, undo.
2. **Do not repeat yourself:** skip a letter this seat has already tried (`printUnique`); Make 24 Task B returns *one expression*.
3. **An expression is a tree.** The root is the *outermost* operation, done after its operands are evaluated.
4. **Read the tree** in three orders: pre-, in-, and post-order. They differ by *one line* of code; in-order with parentheses adds two more.

Roadmap: One Idea in Four Steps

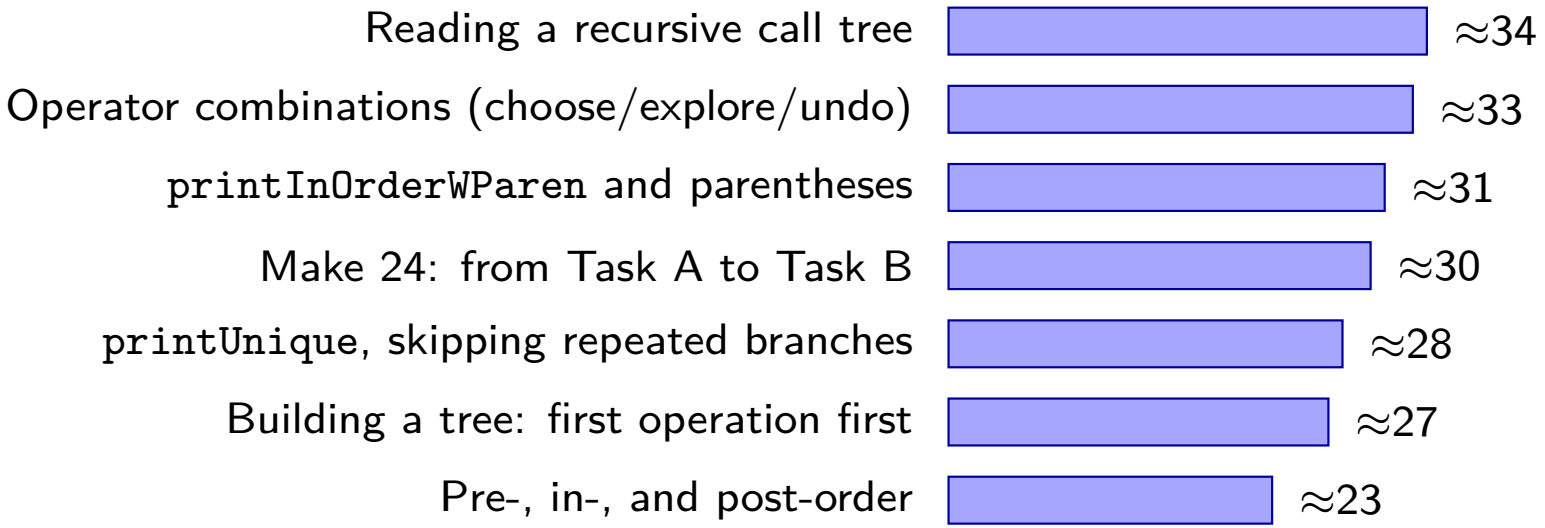
Class 17 was one idea that kept getting a picture added.

1. **Grow an expression** by recursion: choose an operator, explore, undo.
2. **Do not repeat yourself:** skip a letter this seat has already tried (`printUnique`); Make 24 Task B returns *one expression*.
3. **An expression is a tree.** The root is the *outermost* operation, done after its operands are evaluated.
4. **Read the tree** in three orders: pre-, in-, and post-order. They differ by *one line* of code; in-order with parentheses adds two more.

How to use this review. Each slide asks a question first. Try it on paper before you look at its answer.

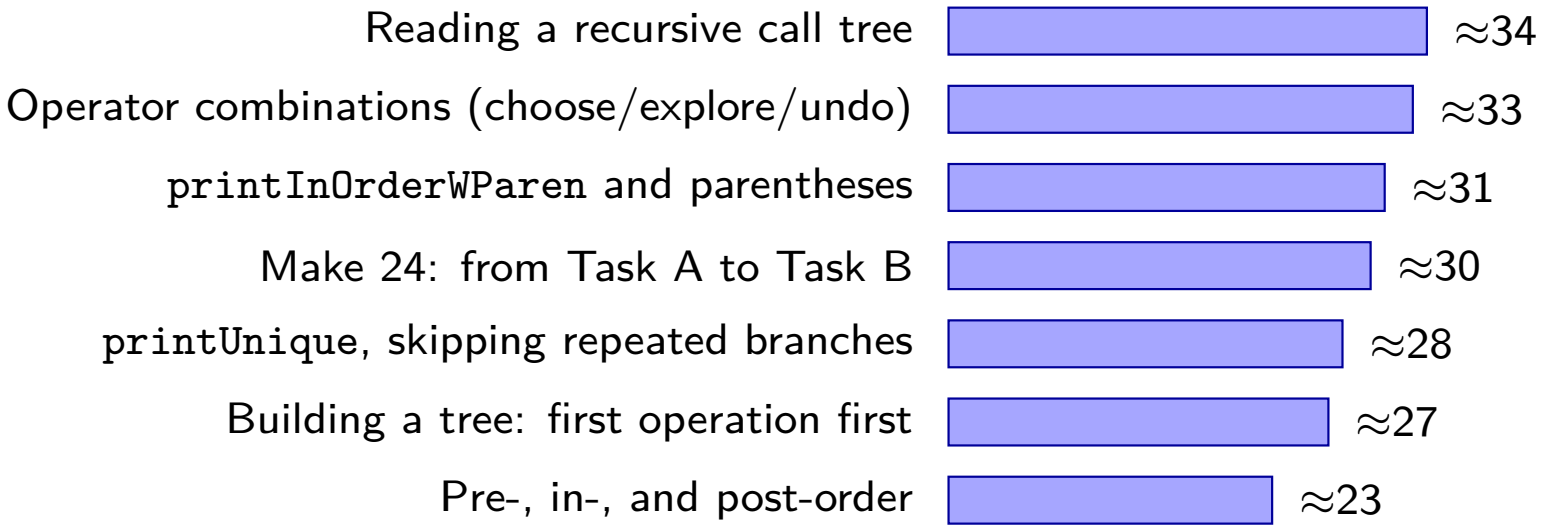
What You Asked For

On the exit slip you checked the topics you wanted clarified (about 65 slips; each student could check several, and counts are approximate). Today's review follows your answers.



What You Asked For

On the exit slip you checked the topics you wanted clarified (about 65 slips; each student could check several, and counts are approximate). Today's review follows your answers.



So the plan: (1) two drills on **reading call trees**; (2) the ops choose-explore-undo shape; (3) `printInOrderWParen` and *where the parentheses come from*; (4) a quick pass over Task B, `printUnique`, trees and the three orders.

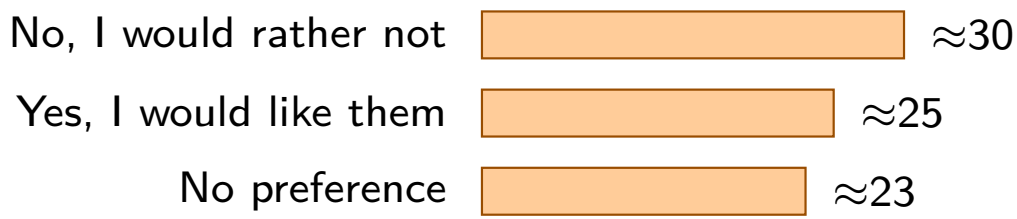
You Also Told Me: Pace and Quick Checks

Two more questions from the exit slips (counts are approximate; some marks were faint).

How was the pace? (about 75 slips)



Would you like pop quizzes? (about 78 marks)



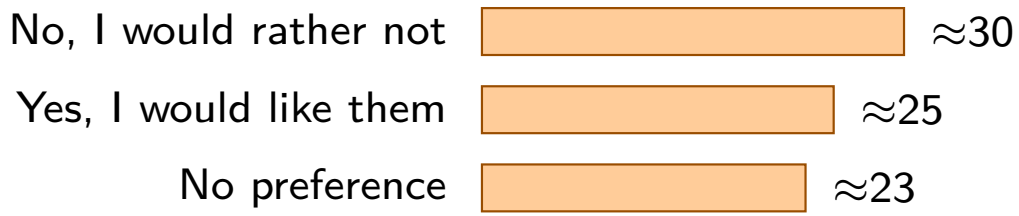
You Also Told Me: Pace and Quick Checks

Two more questions from the exit slips (counts are approximate; some marks were faint).

How was the pace? (about 75 slips)



Would you like pop quizzes? (about 78 marks)



What this means. No respondent selected "too slow," and about 4 in 10 of you asked me to slow down, so we will trace more examples by hand. The class is split on pop quizzes, so any quick check will be short and ungraded.

Drill 1: Read the Call Tree of ops

`v = {2, 5}`, `expr = "2"`, first call `ops(v, 1, expr)`. Each step is one line of the loop body. Fill in `expr` **after** each step, then reveal.

#	Step	Event	`expr` after
1	choose +	append +5; call `ops(v, 2, expr)`	
2	`ops(v, 2, expr)`	`k == v.size()`: print	
3	undo	`expr.resize(len)`, `len = 1`	
4	choose −	append −5; call	
5	`ops(v, 2, expr)`	print	
6	undo	`resize(1)`	
7			

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op; expr += std::to_string(v[k]);   // choose
        ops(v, k + 1, expr);                        // explore
        expr.resize(len);                           // undo
    }
}
```

Drill 1: Read the Call Tree of ops

`v = {2, 5}`, `expr = "2"`, first call `ops(v, 1, expr)`. Each step is one line of the loop body. Fill in `expr` **after** each step, then reveal.

#	**Step**	**Event**	`expr` **after**
1	choose +	append +5; call `ops(v, 2, expr)`	`"2+5"`
2	`ops(v, 2, expr)`	`k == v.size()`: print	
3	undo	`expr.resize(len)`, `len = 1`	
4	choose -	append -5; call	
5	`ops(v, 2, expr)`	print	
6	undo	`resize(1)`	
7			

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op; expr += std::to_string(v[k]);   // choose
        ops(v, k + 1, expr);                        // explore
        expr.resize(len);                           // undo
    }
}
```

Drill 1: Read the Call Tree of ops

`v = {2, 5}`, `expr = "2"`, first call `ops(v, 1, expr)`. Each step is one line of the loop body. Fill in `expr` **after** each step, then reveal.

#	**Step**	**Event**	`expr` **after**
1	choose +	append +5; call `ops(v, 2, expr)`	`"2+5"`
2	`ops(v, 2, expr)`	`k == v.size()`: print	`"2+5"` (prints 2+5)
3	undo	`expr.resize(len)`, `len = 1`	
4	choose −	append −5; call	
5	`ops(v, 2, expr)`	print	
6	undo	`resize(1)`	
7			

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op; expr += std::to_string(v[k]);   // choose
        ops(v, k + 1, expr);                        // explore
        expr.resize(len);                           // undo
    }
}
```

Drill 1: Read the Call Tree of ops

`v = {2, 5}`, `expr = "2"`, first call `ops(v, 1, expr)`. Each step is one line of the loop body. Fill in `expr` **after** each step, then reveal.

#	**Step**	**Event**	`expr` **after**
1	choose +	append +5; call `ops(v, 2, expr)`	`"2+5"`
2	`ops(v, 2, expr)`	`k == v.size()`: print	`"2+5"` (prints 2+5)
3	undo	`expr.resize(len)`, `len = 1`	`"2"`
4	choose -	append -5; call	
5	`ops(v, 2, expr)`	print	
6	undo	`resize(1)`	
7			

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op; expr += std::to_string(v[k]);   // choose
        ops(v, k + 1, expr);                        // explore
        expr.resize(len);                           // undo
    }
}
```

Drill 1: Read the Call Tree of ops

`v = {2, 5}`, `expr = "2"`, first call `ops(v, 1, expr)`. Each step is one line of the loop body. Fill in `expr` **after** each step, then reveal.

#	**Step**	**Event**	`expr` **after**
1	choose +	append +5; call `ops(v, 2, expr)`	`"2+5"`
2	`ops(v, 2, expr)`	`k == v.size()`: print	`"2+5"` (prints 2+5)
3	undo	`expr.resize(len)`, `len = 1`	`"2"`
4	choose -	append -5; call	`"2-5"`
5	`ops(v, 2, expr)`	print	
6	undo	`resize(1)`	
7			

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op; expr += std::to_string(v[k]);   // choose
        ops(v, k + 1, expr);                        // explore
        expr.resize(len);                           // undo
    }
}
```

Drill 1: Read the Call Tree of ops

`v = {2, 5}`, `expr = "2"`, first call `ops(v, 1, expr)`. Each step is one line of the loop body. Fill in `expr` **after** each step, then reveal.

#	**Step**	**Event**	`expr` **after**
1	choose +	append +5; call `ops(v, 2, expr)`	`"2+5"`
2	`ops(v, 2, expr)`	`k == v.size()`: print	`"2+5"` (prints 2+5)
3	undo	`expr.resize(len)`, `len = 1`	`"2"`
4	choose -	append -5; call	`"2-5"`
5	`ops(v, 2, expr)`	print	`"2-5"` (prints 2-5)
6	undo	`resize(1)`	
7			

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op; expr += std::to_string(v[k]);   // choose
        ops(v, k + 1, expr);                        // explore
        expr.resize(len);                           // undo
    }
}
```

Drill 1: Read the Call Tree of ops

`v = {2, 5}`, `expr = "2"`, first call `ops(v, 1, expr)`. Each step is one line of the loop body. Fill in `expr` **after** each step, then reveal.

#	**Step**	**Event**	`expr` **after**
1	choose +	append +5; call `ops(v, 2, expr)`	`"2+5"`
2	`ops(v, 2, expr)`	`k == v.size()`: print	`"2+5"` (prints 2+5)
3	undo	`expr.resize(len)`, `len = 1`	`"2"`
4	choose -	append -5; call	`"2-5"`
5	`ops(v, 2, expr)`	print	`"2-5"` (prints 2-5)
6	undo	`resize(1)`	`"2"`
7			

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op; expr += std::to_string(v[k]);   // choose
        ops(v, k + 1, expr);                        // explore
        expr.resize(len);                           // undo
    }
}
```

Drill 1: Read the Call Tree of ops

`v = {2, 5}`, `expr = "2"`, first call `ops(v, 1, expr)`. Each step is one line of the loop body. Fill in `expr` **after** each step, then reveal.

#	Step	Event	`expr` **after**
1	choose +	append +5; call `ops(v, 2, expr)`	`"2+5"`
2	`ops(v, 2, expr)`	`k == v.size()`: print	`"2+5"` (prints 2+5)
3	undo	`expr.resize(len)`, `len = 1`	`"2"`
4	choose −	append −5; call	`"2-5"`
5	`ops(v, 2, expr)`	print	`"2-5"` (prints 2−5)
6	undo	`resize(1)`	`"2"`
7	. . . the same for * and /: prints 2*5 and 2/5; the final `expr` is `"2"`.		

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op; expr += std::to_string(v[k]);   // choose
        ops(v, k + 1, expr);                        // explore
        expr.resize(len);                           // undo
    }
}
```

Drill 1: Read the Call Tree of ops

`v = {2, 5}`, `expr = "2"`, first call `ops(v, 1, expr)`. Each step is one line of the loop body. Fill in `expr` **after** each step, then reveal.

#	**Step**	**Event**	`expr` **after**
1	choose +	append +5; call `ops(v, 2, expr)`	`"2+5"`
2	`ops(v, 2, expr)`	`k == v.size()`: print	`"2+5"` (prints 2+5)
3	undo	`expr.resize(len)`, `len = 1`	`"2"`
4	choose -	append -5; call	`"2-5"`
5	`ops(v, 2, expr)`	print	`"2-5"` (prints 2-5)
6	undo	`resize(1)`	`"2"`
7	. . . the same for * and /: prints 2*5 and 2/5; the final `expr` is `"2"`.		

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op; expr += std::to_string(v[k]);   // choose
        ops(v, k + 1, expr);                        // explore
        expr.resize(len);                           // undo
    }
}
```

Count. 4 lines and $1 + 4 = 5$ calls. Every **choose** is matched by an **undo**, so `expr` is again `"2"` each time control returns to the first call.

Drill 2: Read the Call Tree of `printAll("AB", 0)`

Each box is one call `printAll(s, k)` with `s` **when the call starts**. The label on an arrow is the swap done just before that call. A green box prints.

```
void printAll(std::string& s, int k) {
  if (k == (int)s.size()) {
    std::cout << s << "\n"; return;
  }
  for (int i = k; i < (int)s.size(); i++) {
    std::swap(s[k], s[i]); // choose
    printAll(s, k + 1);    // explore
    std::swap(s[k], s[i]); // undo
  }
}
```

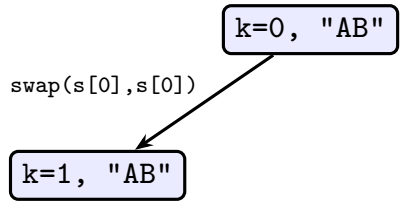
`k=0, "AB"`

Q. Predict the tree before you reveal it: how many calls, and what is printed?

Drill 2: Read the Call Tree of `printAll("AB", 0)`

Each box is one call `printAll(s, k)` with `s` **when the call starts**. The label on an arrow is the swap done just before that call. A green box prints.

```
void printAll(std::string& s, int k) {
  if (k == (int)s.size()) {
    std::cout << s << "\n"; return;
  }
  for (int i = k; i < (int)s.size(); i++) {
    std::swap(s[k], s[i]); // choose
    printAll(s, k + 1);    // explore
    std::swap(s[k], s[i]); // undo
  }
}
```

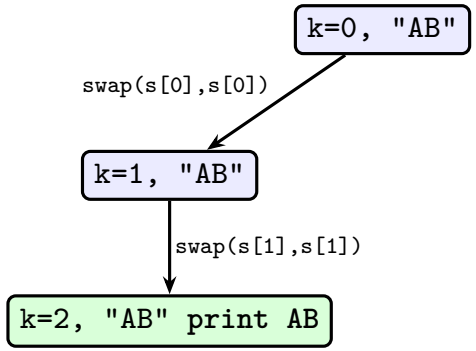


Call 1 loops over `i = 0, 1`. First `i = 0`: swapping a letter with itself does nothing.

Drill 2: Read the Call Tree of `printAll("AB", 0)`

Each box is one call `printAll(s, k)` with `s` **when the call starts**. The label on an arrow is the swap done just before that call. A green box prints.

```
void printAll(std::string& s, int k) {
  if (k == (int)s.size()) {
    std::cout << s << "\n"; return;
  }
  for (int i = k; i < (int)s.size(); i++) {
    std::swap(s[k], s[i]); // choose
    printAll(s, k + 1);    // explore
    std::swap(s[k], s[i]); // undo
  }
}
```

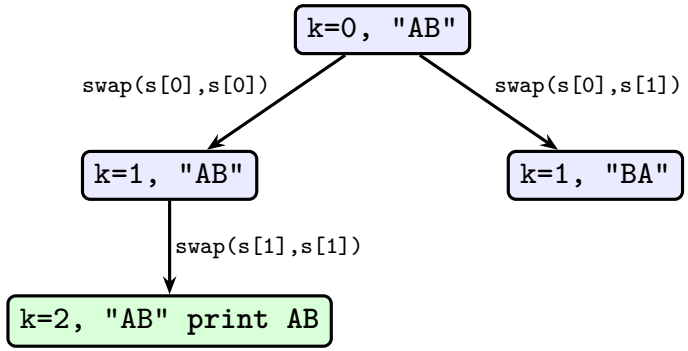


Call 1 loops over `i = 0, 1`. First `i = 0`: swapping a letter with itself does nothing.

Drill 2: Read the Call Tree of `printAll("AB", 0)`

Each box is one call `printAll(s, k)` with `s` **when the call starts**. The label on an arrow is the swap done just before that call. A green box prints.

```
void printAll(std::string& s, int k) {
  if (k == (int)s.size()) {
    std::cout << s << "\n"; return;
  }
  for (int i = k; i < (int)s.size(); i++) {
    std::swap(s[k], s[i]); // choose
    printAll(s, k + 1);    // explore
    std::swap(s[k], s[i]); // undo
  }
}
```

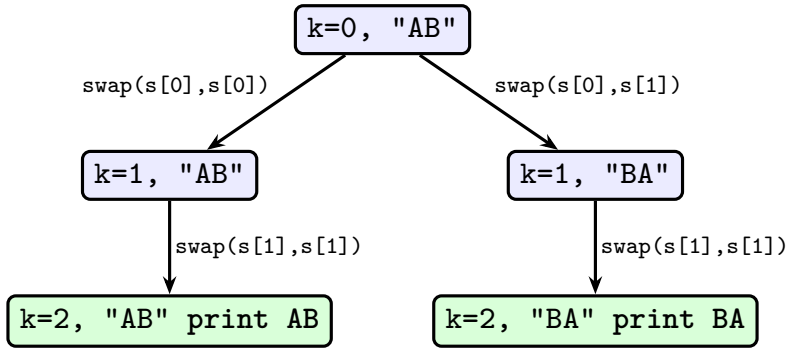


Call 1 loops over `i = 0, 1`. First `i = 0`: swapping a letter with itself does nothing.

Drill 2: Read the Call Tree of `printAll("AB", 0)`

Each box is one call `printAll(s, k)` with `s` **when the call starts**. The label on an arrow is the swap done just before that call. A green box prints.

```
void printAll(std::string& s, int k) {
  if (k == (int)s.size()) {
    std::cout << s << "\n"; return;
  }
  for (int i = k; i < (int)s.size(); i++) {
    std::swap(s[k], s[i]); // choose
    printAll(s, k + 1);    // explore
    std::swap(s[k], s[i]); // undo
  }
}
```



Answer. 5 calls $(1 + 2 + 2)$, 2 lines: `AB` then `BA`. After the whole call returns, the undo swaps have restored `s` to `"AB"`.

Review 1: Choose, Explore, Undo in ops

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op; expr += std::to_string(v[k]);   // choose
        ops(v, k + 1, expr);                        // explore
        expr.resize(len);                           // undo
    }
}
```

Q1. For `v = {4, 5, 6, 7}`, `expr = "4"`, and the call `ops(v, 1, expr)`: how many lines are printed? How many calls are made?

Q2. What is `expr` just after `expr.resize(len)` when `len = 3` and `expr = "1+2*3"`?

Note. ops inserts operators between the numbers in their original order and prints expression *text*; it does not evaluate or search parenthesizations.

Review 1: Choose, Explore, Undo in ops

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op; expr += std::to_string(v[k]);   // choose
        ops(v, k + 1, expr);                        // explore
        expr.resize(len);                           // undo
    }
}
```

Q1. For `v = {4, 5, 6, 7}`, `expr = "4"`, and the call `ops(v, 1, expr)`: how many lines are printed? How many calls are made?

Answer. Three gaps, four operators each: $4 \times 4 \times 4 = \mathbf{64}$ lines. Calls: $1 + 4 + 16 + 64 = \mathbf{85}$. The last gap changes fastest.

Q2. What is `expr` just after `expr.resize(len)` when `len = 3` and `expr = "1+2*3"`?

Note. ops inserts operators between the numbers in their original order and prints expression *text*; it does not evaluate or search parenthesizations.

Review 1: Choose, Explore, Undo in ops

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();
        expr += op; expr += std::to_string(v[k]);   // choose
        ops(v, k + 1, expr);                        // explore
        expr.resize(len);                           // undo
    }
}
```

Q1. For `v = {4, 5, 6, 7}`, `expr = "4"`, and the call `ops(v, 1, expr)`: how many lines are printed? How many calls are made?

Answer. Three gaps, four operators each: $4 \times 4 \times 4 = \mathbf{64}$ lines. Calls: $1 + 4 + 16 + 64 = \mathbf{85}$. The last gap changes fastest.

Q2. What is `expr` just after `expr.resize(len)` when `len = 3` and `expr = "1+2*3"`? **Answer:** `"1+2"`. The undo cuts the string back to the length it had *before* the choose, so the next operator starts from the same string.

Note. ops inserts operators between the numbers in their original order and prints expression *text*; it does not evaluate or search parenthesizations.

The Two Permutation Functions

printAll

```
void printAll(std::string& s, int k) {
  if (k == (int)s.size()) {
    std::cout << s << "\n"; return;
  }
  for (int i = k; i < (int)s.size(); i++) {
    std::swap(s[k], s[i]); // choose
    printAll(s, k + 1);    // explore
    std::swap(s[k], s[i]); // undo
  }
}
```

printUnique

```
void printUnique(std::string& s, int k) {
  if (k == (int)s.size()) {
    std::cout << s << "\n"; return;
  }
  std::set<char> tried;
  for (int i = k; i < (int)s.size(); i++) {
    if (!tried.insert(s[i]).second) {
          continue; }
    std::swap(s[k], s[i]);
    printUnique(s, k + 1);
    std::swap(s[k], s[i]);
  }
}
```

How to count. Count every function invocation, including the initial call and the base-case calls. The counts below and in the following review assume the base case `k == s.size()`. Read `printAll("AB", 0)` as shorthand for:

```
std::string s = "AB";

printAll(s, 0);
```

Review 2: `printAll` vs. `printUnique`

Q1. For `s = "ABB"`, `printAll(s, 0)` prints 6 lines but only 3 different strings. Why?

Q2. What does `std::set<char> tried` fix, and why is it declared **inside** the function?

Q3. Fill in the numbers.

	calls	**lines printed**
`printAll("ABC")`	?	?
`printAll("ABB")`	?	?
`printUnique("ABB")`	?	?

Review 2: `printAll` vs. `printUnique`

Q1. For `s = "ABB"`, `printAll(s, 0)` prints 6 lines but only 3 different strings. Why? **Answer.** It treats the two B's as different people, so each string appears twice.

Q2. What does `std::set<char> tried` fix, and why is it declared **inside** the function?

Q3. Fill in the numbers.

	calls	**lines printed**
`printAll("ABC")`	?	?
`printAll("ABB")`	?	?
`printUnique("ABB")`	?	?

Review 2: `printAll` vs. `printUnique`

Q1. For `s = "ABB"`, `printAll(s, 0)` prints 6 lines but only 3 different strings. Why? **Answer.** It treats the two B's as different people, so each string appears twice.

Q2. What does `std::set<char> tried` fix, and why is it declared **inside** the function? **Answer.** Each call (each seat) remembers the letters *it* has already tried and skips a repeat. Inside the function, every call starts with an empty set; a global set would remember letters from other seats and wrongly skip good branches.

Q3. Fill in the numbers.

	calls	**lines printed**
`printAll("ABC")`	?	?
`printAll("ABB")`	?	?
`printUnique("ABB")`	?	?

Review 2: `printAll` vs. `printUnique`

Q1. For `s = "ABB"`, `printAll(s, 0)` prints 6 lines but only 3 different strings. Why? **Answer.** It treats the two B's as different people, so each string appears twice.

Q2. What does `std::set<char> tried` fix, and why is it declared **inside** the function? **Answer.** Each call (each seat) remembers the letters *it* has already tried and skips a repeat. Inside the function, every call starts with an empty set; a global set would remember letters from other seats and wrongly skip good branches.

Q3. Fill in the numbers.

	calls	**lines printed**
`printAll("ABC")`	16	6
`printAll("ABB")`	16	6
`printUnique("ABB")`	**9**	**3**

Review 2: `printAll` vs. `printUnique`

Q1. For `s = "ABB"`, `printAll(s, 0)` prints 6 lines but only 3 different strings. Why? **Answer.** It treats the two B's as different people, so each string appears twice.

Q2. What does `std::set<char> tried` fix, and why is it declared **inside** the function? **Answer.** Each call (each seat) remembers the letters *it* has already tried and skips a repeat. Inside the function, every call starts with an empty set; a global set would remember letters from other seats and wrongly skip good branches.

Q3. Fill in the numbers.

	calls	**lines printed**
`printAll("ABC")`	16	6
`printAll("ABB")`	16	6
`printUnique("ABB")`	**9**	**3**

Skip: `if (!tried.insert(s[i]).second) { continue; }` (`.second` is `true` only if the letter was new)

Review 3: Make 24, Task A and Task B

Q1. Task B is Task A with a change. What stays the same, and what extra information does Task B carry?

Example. Values 3 and 4 combine into 7, and their texts combine into `"(3 + 4)"`: a new composite expression keeps its grouping by adding parentheses.

Review 3: Make 24, Task A and Task B

Q1. Task B is Task A with a change. What stays the same, and what extra information does Task B carry?

Answer. Same: the search (the loops, swap-and-pop removal, choose-explore-undo). **Different:** each number in the hand now carries its *text* as well as its value (`Expression_Value`), and a call returns `std::string` instead of `bool`.

	Task A	**Task B**
Hand	`vector<double>`	`vector<Expression_Value>` (value *and* text)
Returns	`bool`	`std::string`
"Found nothing here"	`false`	empty string
A child found one	return `true`	return that child's string

Example. Values 3 and 4 combine into 7, and their texts combine into `"(3 + 4)"`: a new composite expression keeps its grouping by adding parentheses.

Review 3 (continued): Task B Search Rules

Q2. In Task B, why must a call keep looping if a child returns the empty string?

Review 3 (continued): Task B Search Rules

Q2. In Task B, why must a call keep looping if a child returns the empty string?

Review 3 (continued): Task B Search Rules

Q2. In Task B, why must a call keep looping if a child returns the empty string? **Answer.** Empty means "nothing found in *that* branch", not "nothing exists". The next choice may still succeed. The call returns as soon as a child returns a *non-empty* string.

Review 3 (continued): Task B Search Rules

Q2. In Task B, why must a call keep looping if a child returns the empty string? **Answer.** Empty means "nothing found in *that* branch", not "nothing exists". The next choice may still succeed. The call returns as soon as a child returns a *non-empty* string.

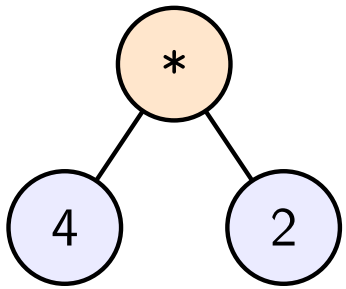
Also remember the parts of the assignment that do not change: the single-value base case (one number left), the rule for division by zero, and the floating-point comparison for the target 24.

Review 4: An Expression Is a Tree

Rule: first operation first. Build the tree in the order the operations are *performed*. The **root** is the outermost operation, performed after its operand subexpressions have been evaluated (not the rightmost symbol).

`3 + 4 * 2`

`8 - 3 - 2`

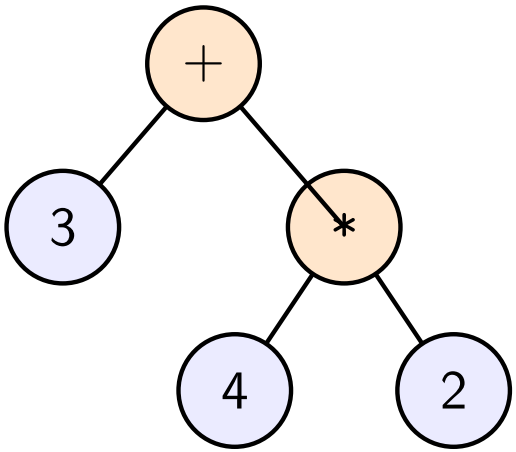


Q. In `3 + 4 * 2`, which operation happens first?

Review 4: An Expression Is a Tree

Rule: first operation first. Build the tree in the order the operations are *performed*. The **root** is the outermost operation, performed after its operand subexpressions have been evaluated (not the rightmost symbol).

`3 + 4 * 2`



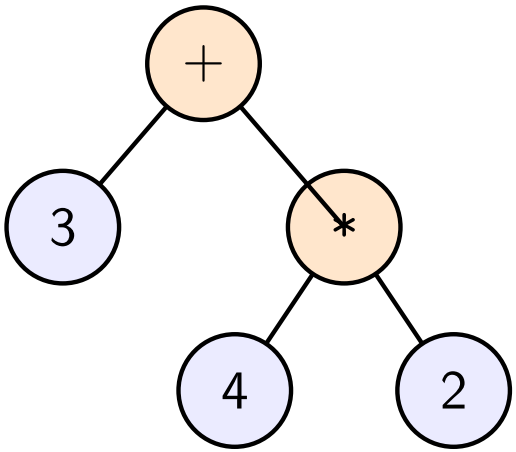
`8 - 3 - 2`

Answer. $4 * 2$ first (multiplication binds tighter), so + is last: the **root**.

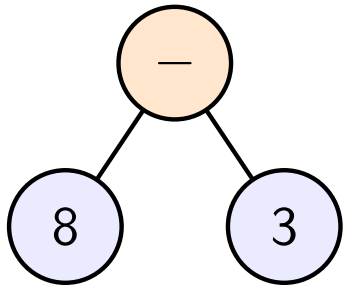
Review 4: An Expression Is a Tree

Rule: first operation first. Build the tree in the order the operations are *performed*. The **root** is the outermost operation, performed after its operand subexpressions have been evaluated (not the rightmost symbol).

`3 + 4 * 2`



`8 - 3 - 2`

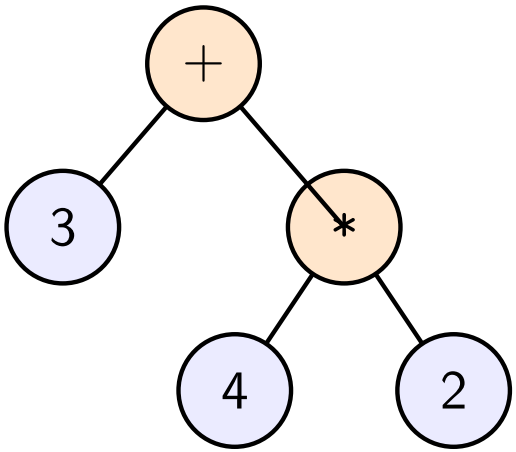


Q. In `8 - 3 - 2`, a tie: which `-` happens first?

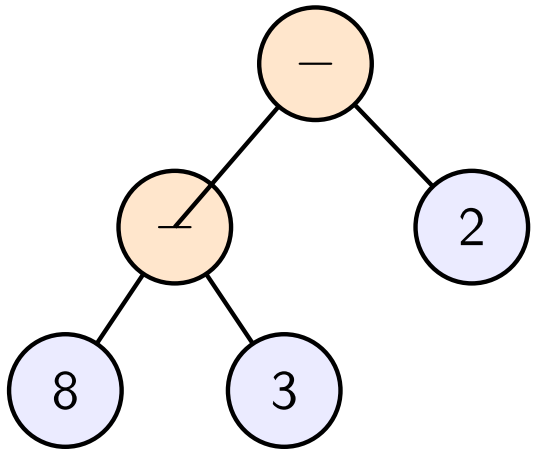
Review 4: An Expression Is a Tree

Rule: first operation first. Build the tree in the order the operations are *performed*. The **root** is the outermost operation, performed after its operand subexpressions have been evaluated (not the rightmost symbol).

`3 + 4 * 2`



`8 - 3 - 2`

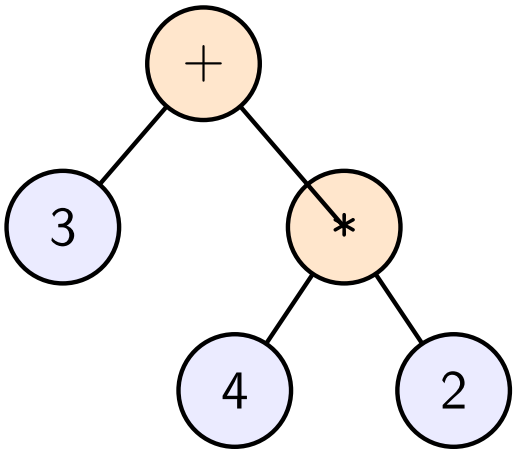


Answer. For the binary operators `+ - * /` used here, equal-precedence operators associate left to right: $8 - 3$ first, so the *second* `-` is the root. Leaves are numbers; every operator has two subtrees.

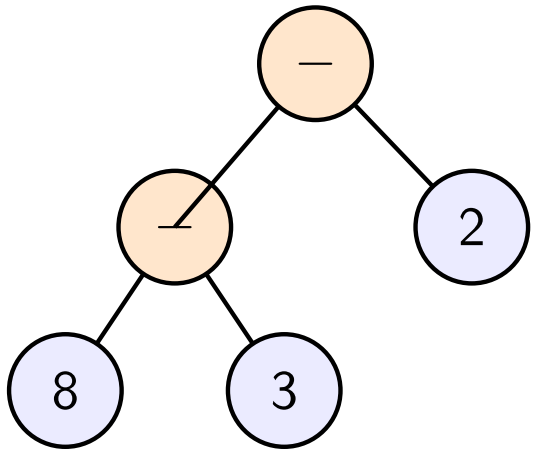
Review 4: An Expression Is a Tree

Rule: first operation first. Build the tree in the order the operations are *performed*. The **root** is the outermost operation, performed after its operand subexpressions have been evaluated (not the rightmost symbol).

`3 + 4 * 2`



`8 - 3 - 2`

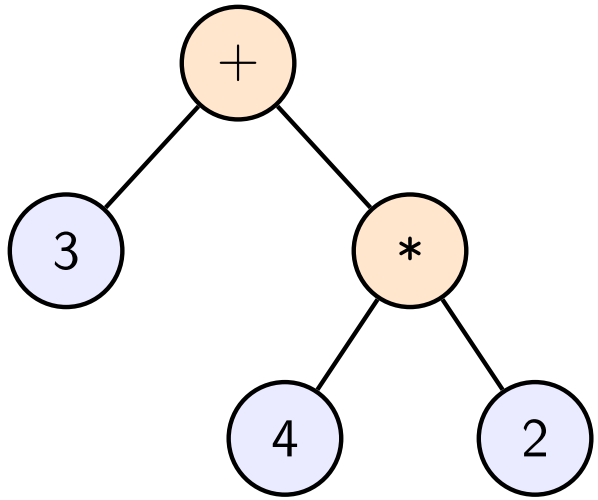


Summary. `3 + 4 * 2`: multiplication groups first, so + is the root.
`8 - 3 - 2`: subtraction associates left to right, so the second - is the root.
For the binary operators used here, every operator node has two children; numbers are leaves.

Review 5: Pre-, In-, and Post-Order

Rule. The words **pre**, **in**, **post** say where the **root** goes. **The left subtree always comes before the right.**

- **Pre-order: root**, left, right
- **In-order:** left, **root**, right
- **Post-order:** left, right, **root**



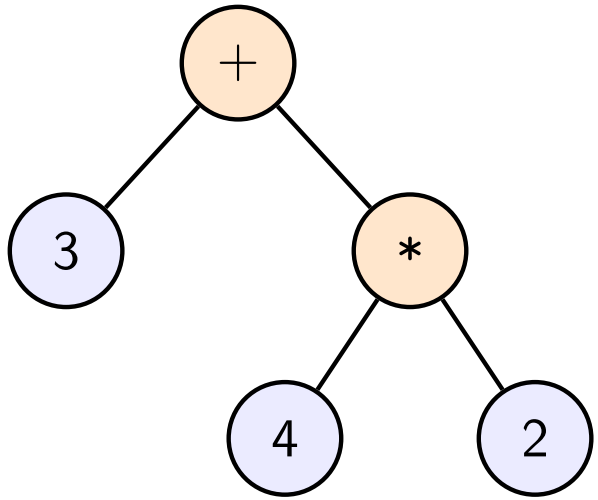
Every subtree is read by the **same rule**: that is recursion.

Q. Read the tree in all three orders.

Review 5: Pre-, In-, and Post-Order

Rule. The words **pre**, **in**, **post** say where the **root** goes. **The left subtree always comes before the right.**

- **Pre-order: root**, left, right
- **In-order:** left, **root**, right
- **Post-order:** left, right, **root**



Every subtree is read by the **same rule**: that is recursion.

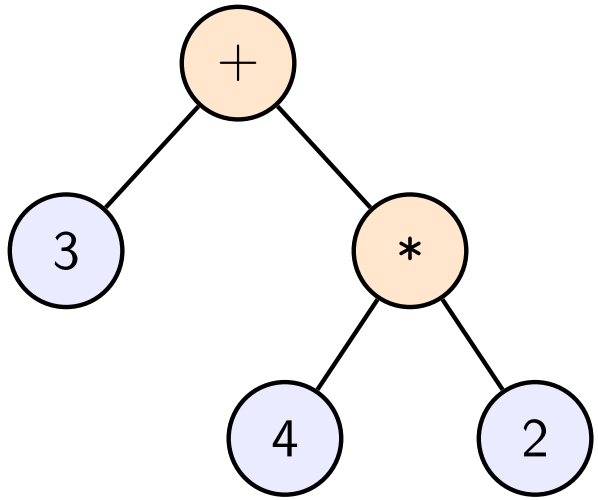
Q. Read the tree in all three orders.

Pre-order: `+ 3 * 4 2`

Review 5: Pre-, In-, and Post-Order

Rule. The words **pre**, **in**, **post** say where the **root** goes. **The left subtree always comes before the right.**

- **Pre-order: root**, left, right
- **In-order:** left, **root**, right
- **Post-order:** left, right, **root**



Every subtree is read by the **same rule**: that is recursion.

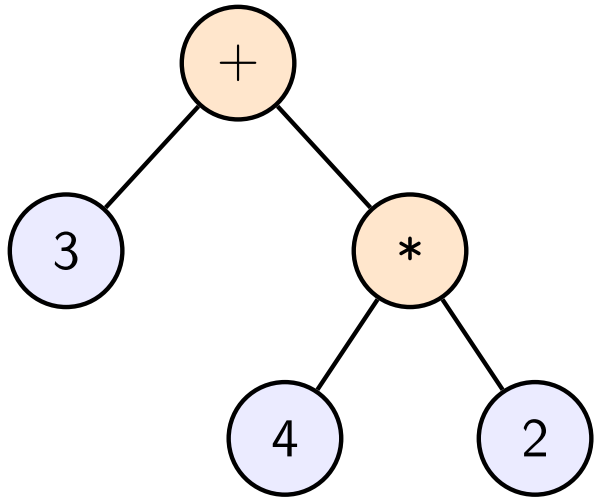
Q. Read the tree in all three orders.

Pre-order: `+ 3 * 4 2`
In-order: `3 + 4 * 2`

Review 5: Pre-, In-, and Post-Order

Rule. The words **pre**, **in**, **post** say where the **root** goes. **The left subtree always comes before the right.**

- **Pre-order: root**, left, right
- **In-order:** left, **root**, right
- **Post-order:** left, right, **root**



Every subtree is read by the **same rule**: that is recursion.

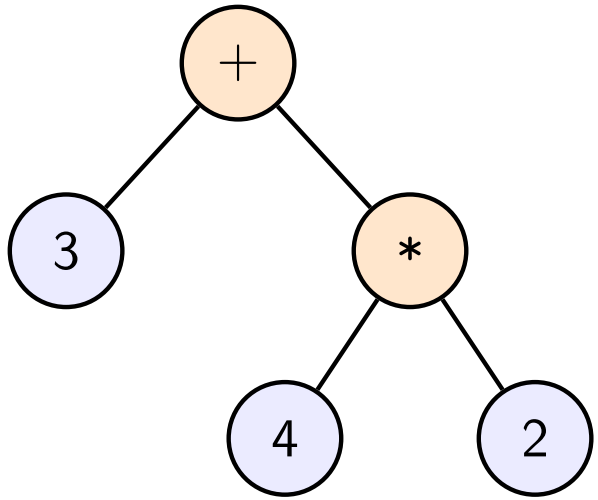
Q. Read the tree in all three orders.

Pre-order: `+ 3 * 4 2`
In-order: `3 + 4 * 2`
Post-order: `3 4 2 * +`

Review 5: Pre-, In-, and Post-Order

Rule. The words **pre**, **in**, **post** say where the **root** goes. **The left subtree always comes before the right.**

- **Pre-order: root**, left, right
- **In-order:** left, **root**, right
- **Post-order:** left, right, **root**



Every subtree is read by the **same rule**: that is recursion.

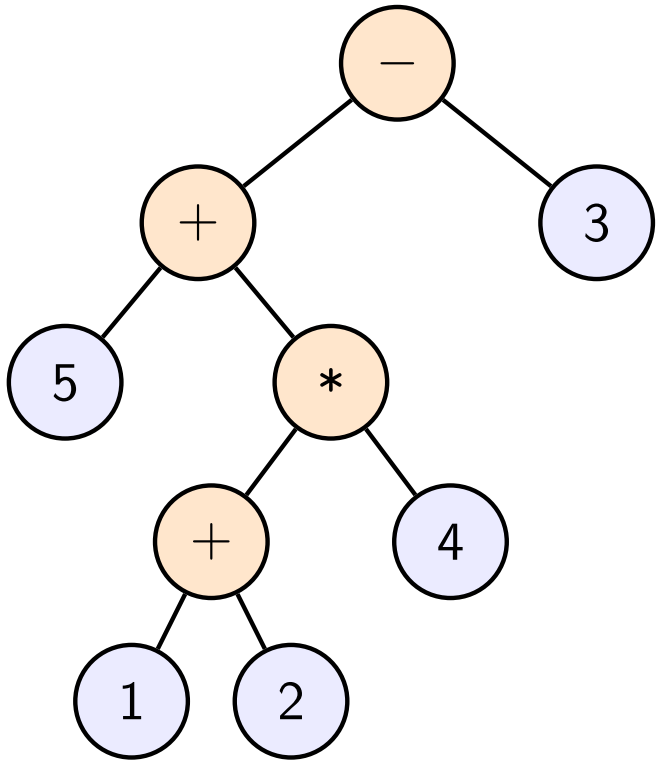
Q. Read the tree in all three orders.

Pre-order: `+ 3 * 4 2`
In-order: `3 + 4 * 2`
Post-order: `3 4 2 * +`

Check. In-order matches here only because no parentheses were needed. In general plain in-order **drops** them (next slides).

Review 6: Why Plain In-Order Is Not Enough

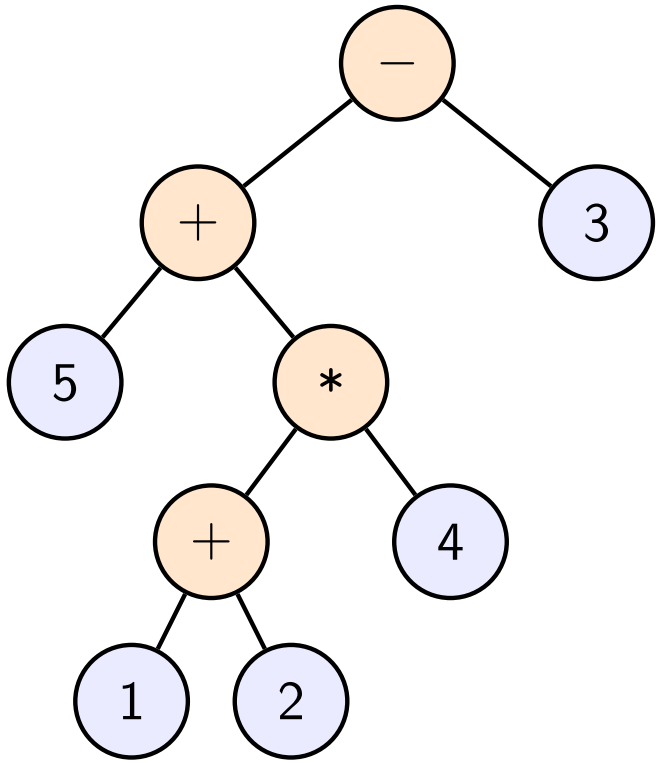
Tree for `5 + (1 + 2) * 4 - 3`.
The last operation is the final −.



Review 6: Why Plain In-Order Is Not Enough

Tree for `5 + (1 + 2) * 4 - 3`.
The last operation is the final −.

Pre-order:
`- + 5 * + 1 2 4 3`

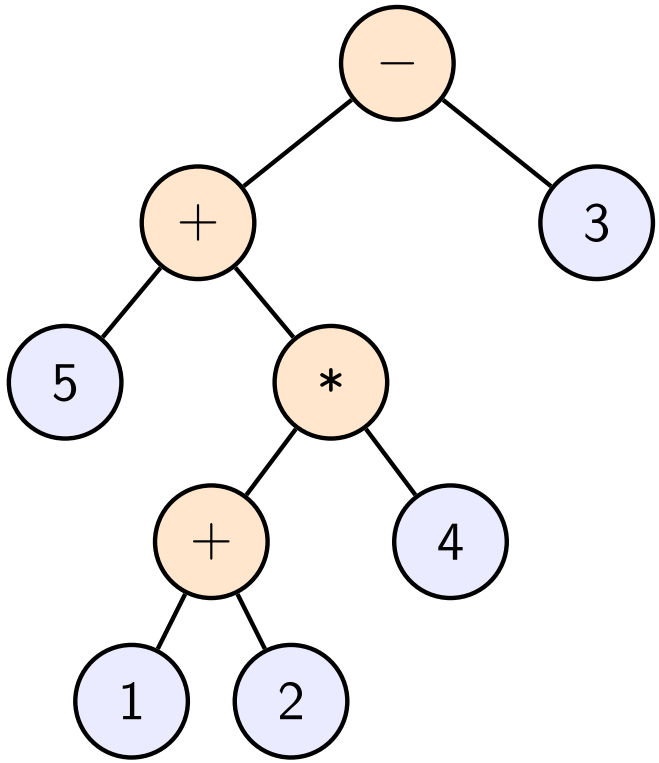


Review 6: Why Plain In-Order Is Not Enough

Tree for `5 + (1 + 2) * 4 - 3`.
The last operation is the final −.

Pre-order:
`- + 5 * + 1 2 4 3`

In-order (no parentheses):
`5 + 1 + 2 * 4 - 3`



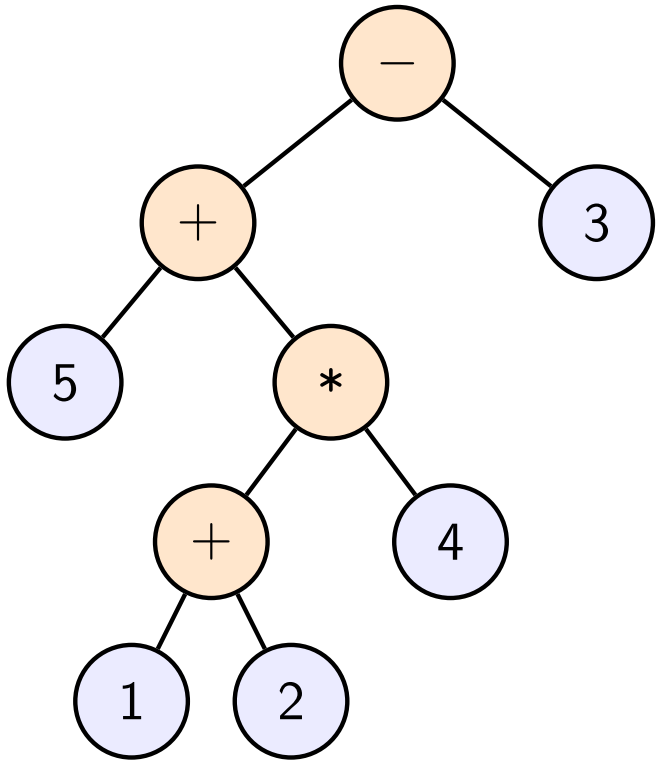
Review 6: Why Plain In-Order Is Not Enough

Tree for `5 + (1 + 2) * 4 - 3`.
The last operation is the final −.

Pre-order:
`- + 5 * + 1 2 4 3`

In-order (no parentheses):
`5 + 1 + 2 * 4 - 3`

Post-order:
`5 1 2 + 4 * + 3 -`



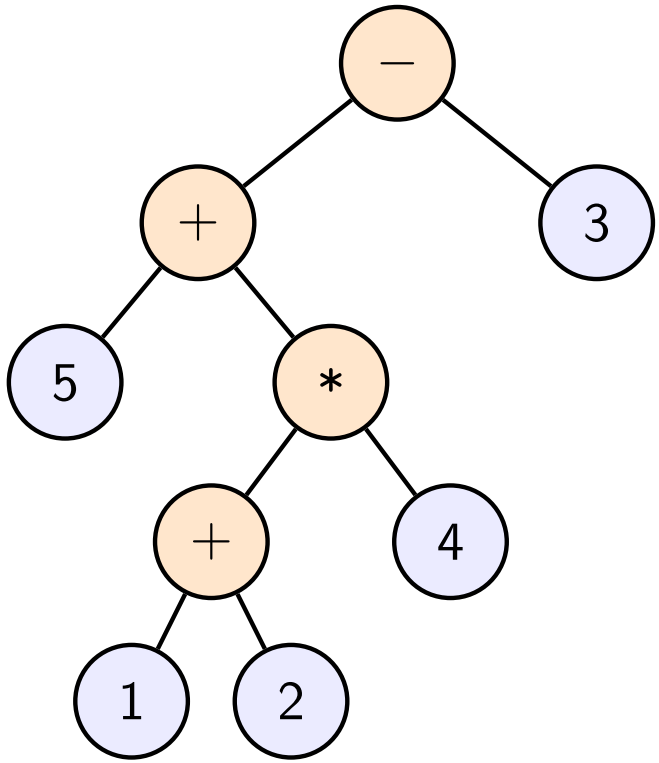
Review 6: Why Plain In-Order Is Not Enough

Tree for `5 + (1 + 2) * 4 - 3`.
The last operation is the final −.

Pre-order:
`- + 5 * + 1 2 4 3`

In-order (no parentheses):
`5 + 1 + 2 * 4 - 3`

Post-order:
`5 1 2 + 4 * + 3 -`



Q. Read the plain in-order line as ordinary arithmetic. What value do you get? What does the tree give?

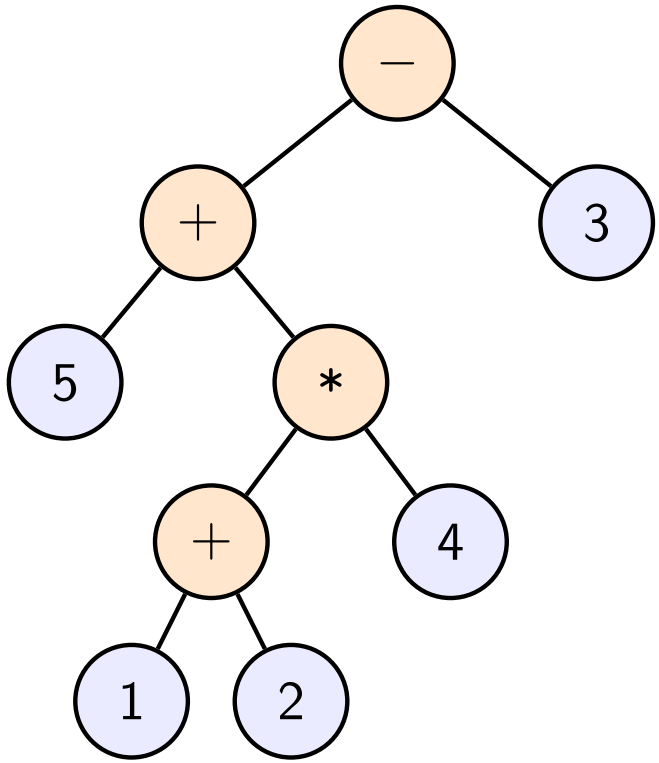
Review 6: Why Plain In-Order Is Not Enough

Tree for `5 + (1 + 2) * 4 - 3`.
The last operation is the final −.

Pre-order:
`- + 5 * + 1 2 4 3`

In-order (no parentheses):
`5 + 1 + 2 * 4 - 3`

Post-order:
`5 1 2 + 4 * + 3 -`



Q. Read the plain in-order line as ordinary arithmetic. What value do you get? What does the tree give?
Answer. The line reads $5 + 1 + 2 \cdot 4 - 3 = 11$, but the tree evaluates to $(1 + 2) \cdot 4 = 12$, then $5 + 12 = 17$, then $17 - 3 = \mathbf{14}$. The shape of the tree carried information that plain in-order loses.

Review 7: One Line Makes the Difference

`struct Node { std::string token; Node* left; Node* right; };` (`print` writes the token.)

preorder

```
void preorder(const Node* r) {
  if (r == nullptr) return;
  print(r->token);
  preorder(r->left);
  preorder(r->right);
}
```

inorder

```
void inorder(const Node* r) {
  if (r == nullptr) return;
  inorder(r->left);
  print(r->token);
  inorder(r->right);
}
```

postorder

```
void postorder(const Node* r) {
  if (r == nullptr) return;
  postorder(r->left);
  postorder(r->right);
  print(r->token);
}
```

Q1. What is the base case, and why is a call on `nullptr` not an error?

Q2. Why does each non-empty call make exactly two recursive calls?

Q3. The three functions differ in exactly one place. Where?

Review 7: One Line Makes the Difference

`struct Node { std::string token; Node* left; Node* right; };` (`print` writes the token.)

preorder

```
void preorder(const Node* r) {
  if (r == nullptr) return;
  print(r->token);
  preorder(r->left);
  preorder(r->right);
}
```

inorder

```
void inorder(const Node* r) {
  if (r == nullptr) return;
  inorder(r->left);
  print(r->token);
  inorder(r->right);
}
```

postorder

```
void postorder(const Node* r) {
  if (r == nullptr) return;
  postorder(r->left);
  postorder(r->right);
  print(r->token);
}
```

Q1. What is the base case, and why is a call on `nullptr` not an error? **Answer.** `nullptr` means "no child": the subtree is empty, so there is nothing to print and the call returns at once. Every branch of the recursion ends there.

Q2. Why does each non-empty call make exactly two recursive calls?

Q3. The three functions differ in exactly one place. Where?

Review 7: One Line Makes the Difference

`struct Node { std::string token; Node* left; Node* right; };` (`print` writes the token.)

preorder

```
void preorder(const Node* r) {
  if (r == nullptr) return;
  print(r->token);
  preorder(r->left);
  preorder(r->right);
}
```

inorder

```
void inorder(const Node* r) {
  if (r == nullptr) return;
  inorder(r->left);
  print(r->token);
  inorder(r->right);
}
```

postorder

```
void postorder(const Node* r) {
  if (r == nullptr) return;
  postorder(r->left);
  postorder(r->right);
  print(r->token);
}
```

Q1. What is the base case, and why is a call on `nullptr` not an error? **Answer.** `nullptr` means "no child": the subtree is empty, so there is nothing to print and the call returns at once. Every branch of the recursion ends there.

Q2. Why does each non-empty call make exactly two recursive calls? **Answer.** The `Node` has two links, `left` and `right`. Each subtree is a smaller tree, so the recursion goes *down* toward `nullptr`.

Q3. The three functions differ in exactly one place. Where?

Review 7: One Line Makes the Difference

`struct Node { std::string token; Node* left; Node* right; };` (print writes the token.)

preorder

```
void preorder(const Node* r) {
  if (r == nullptr) return;
  print(r->token);
  preorder(r->left);
  preorder(r->right);
}
```

inorder

```
void inorder(const Node* r) {
  if (r == nullptr) return;
  inorder(r->left);
  print(r->token);
  inorder(r->right);
}
```

postorder

```
void postorder(const Node* r) {
  if (r == nullptr) return;
  postorder(r->left);
  postorder(r->right);
  print(r->token);
}
```

Q1. What is the base case, and why is a call on `nullptr` not an error? **Answer.** `nullptr` means "no child": the subtree is empty, so there is nothing to print and the call returns at once. Every branch of the recursion ends there.

Q2. Why does each non-empty call make exactly two recursive calls? **Answer.** The `Node` has two links, `left` and `right`. Each subtree is a smaller tree, so the recursion goes *down* toward `nullptr`.

Q3. The three functions differ in exactly one place. Where? **Answer.** The position of the print line (red): **first**, **middle**, **last**.

Review 8: Tracing `postorder` on `3 + 4 * 2`

Each non-empty call makes two child calls; a call on `nullptr` returns at once. **Red** = where a node is printed. The calls run **depth first**. The rows are **steps**, not calls: rows 6 and 7 resume calls already open, and the calls on `nullptr` are left out (the full run makes 11 calls: 5 on nodes, 6 on `nullptr`).

Step	Call	What happens	Output so far
1	`postorder(+)`	go left	

Review 8: Tracing `postorder` on `3 + 4 * 2`

Each non-empty call makes two child calls; a call on `nullptr` returns at once. **Red** = where a node is printed. The calls run **depth first**. The rows are **steps**, not calls: rows 6 and 7 resume calls already open, and the calls on `nullptr` are left out (the full run makes 11 calls: 5 on nodes, 6 on `nullptr`).

Step	Call	What happens	Output so far
1	`postorder(+)`	go left	
2	`  postorder(3)`	both children are `nullptr`; print 3	3

Review 8: Tracing `postorder` on `3 + 4 * 2`

Each non-empty call makes two child calls; a call on `nullptr` returns at once. **Red** = where a node is printed. The calls run **depth first**. The rows are **steps**, not calls: rows 6 and 7 resume calls already open, and the calls on `nullptr` are left out (the full run makes 11 calls: 5 on nodes, 6 on `nullptr`).

Step	Call	What happens	Output so far
1	`postorder(+)`	go left	
2	`  postorder(3)`	both children are `nullptr`; print 3	3
3	`  postorder(*)`	go left	3

Review 8: Tracing `postorder` on `3 + 4 * 2`

Each non-empty call makes two child calls; a call on `nullptr` returns at once. **Red** = where a node is printed. The calls run **depth first**. The rows are **steps**, not calls: rows 6 and 7 resume calls already open, and the calls on `nullptr` are left out (the full run makes 11 calls: 5 on nodes, 6 on `nullptr`).

Step	Call	What happens	Output so far
1	`postorder(+)`	go left	
2	`  postorder(3)`	both children are `nullptr`; print 3	3
3	`  postorder(*)`	go left	3
4	`    postorder(4)`	print 4	3 4

Review 8: Tracing `postorder` on `3 + 4 * 2`

Each non-empty call makes two child calls; a call on `nullptr` returns at once. **Red** = where a node is printed. The calls run **depth first**. The rows are **steps**, not calls: rows 6 and 7 resume calls already open, and the calls on `nullptr` are left out (the full run makes 11 calls: 5 on nodes, 6 on `nullptr`).

Step	Call	What happens	Output so far
1	`postorder(+)`	go left	
2	`  postorder(3)`	both children are `nullptr`; print 3	`3`
3	`  postorder(*)`	go left	`3`
4	`    postorder(4)`	print 4	`3 4`
5	`    postorder(2)`	print 2	`3 4 2`

Review 8: Tracing `postorder` on `3 + 4 * 2`

Each non-empty call makes two child calls; a call on `nullptr` returns at once. **Red** = where a node is printed. The calls run **depth first**. The rows are **steps**, not calls: rows 6 and 7 resume calls already open, and the calls on `nullptr` are left out (the full run makes 11 calls: 5 on nodes, 6 on `nullptr`).

Step	Call	What happens	Output so far
1	`postorder(+)`	go left	
2	`  postorder(3)`	both children are `nullptr`; print 3	`3`
3	`  postorder(*)`	go left	`3`
4	`    postorder(4)`	print 4	`3 4`
5	`    postorder(2)`	print 2	`3 4 2`
6	back in `postorder(*)`	both subtrees done; print *	`3 4 2 *`

Review 8: Tracing `postorder` on `3 + 4 * 2`

Each non-empty call makes two child calls; a call on `nullptr` returns at once. **Red** = where a node is printed. The calls run **depth first**. The rows are **steps**, not calls: rows 6 and 7 resume calls already open, and the calls on `nullptr` are left out (the full run makes 11 calls: 5 on nodes, 6 on `nullptr`).

Step	Call	What happens	Output so far
1	`postorder(+)`	go left	
2	`  postorder(3)`	both children are `nullptr`; print 3	`3`
3	`  postorder(*)`	go left	`3`
4	`    postorder(4)`	print 4	`3 4`
5	`    postorder(2)`	print 2	`3 4 2`
6	back in `postorder(*)`	both subtrees done; print *	`3 4 2 *`
7	back in `postorder(+)`	both subtrees done; print +	`3 4 2 * +`

Review 8: Tracing `postorder` on `3 + 4 * 2`

Each non-empty call makes two child calls; a call on `nullptr` returns at once. **Red** = where a node is printed. The calls run **depth first**. The rows are **steps**, not calls: rows 6 and 7 resume calls already open, and the calls on `nullptr` are left out (the full run makes 11 calls: 5 on nodes, 6 on `nullptr`).

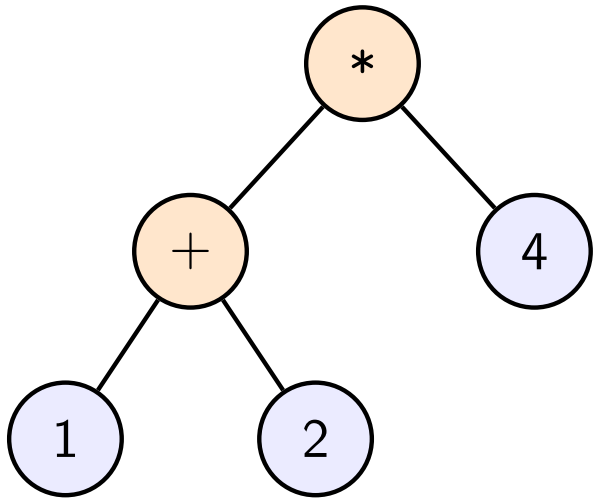
Step	Call	What happens	Output so far
1	`postorder(+)`	go left	
2	`  postorder(3)`	both children are `nullptr`; print 3	`3`
3	`  postorder(*)`	go left	`3`
4	`    postorder(4)`	print 4	`3 4`
5	`    postorder(2)`	print 2	`3 4 2`
6	`  back in postorder(*)`	both subtrees done; print *	`3 4 2 *`
7	`back in postorder(+)`	both subtrees done; print +	`3 4 2 * +`

Q. What would change if the print line moved to the top of the function? **Answer.** The calls follow the same path, but each node prints *on entry*, before its subtrees, so the output grows differently and ends as `+ 3 * 4 2`: pre-order.

Review 9a: Is This Node an Operator?

To add parentheses we must know, at each node, whether it is an **operator** (needs (and)) or a **number** (prints just itself).

Q. Every operator has two children and every number is a leaf. Which test on a non-null `root` says "this node is an operator"?



(A) `root->left == nullptr`

(B) `root->left != nullptr`

(C) `root->token == "+"`

(D) `root->right == nullptr`

Review 9a: Is This Node an Operator?

To add parentheses we must know, at each node, whether it is an **operator** (needs (and)) or a **number** (prints just itself).

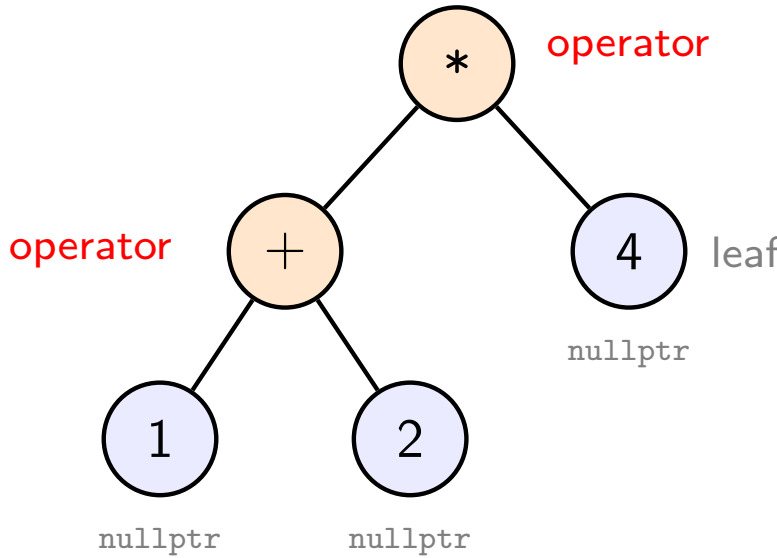
Q. Every operator has two children and every number is a leaf. Which test on a non-null `root` says "this node is an operator"?

(A) `root->left == nullptr`

(B) `root->left != nullptr`

(C) `root->token == "+"`

(D) `root->right == nullptr`



Answer: (B). An operator has children, so `left` is not `nullptr`. A number is a leaf: both links are `nullptr`. (A) and (D) are true only for leaves; (C) tests just one operator.

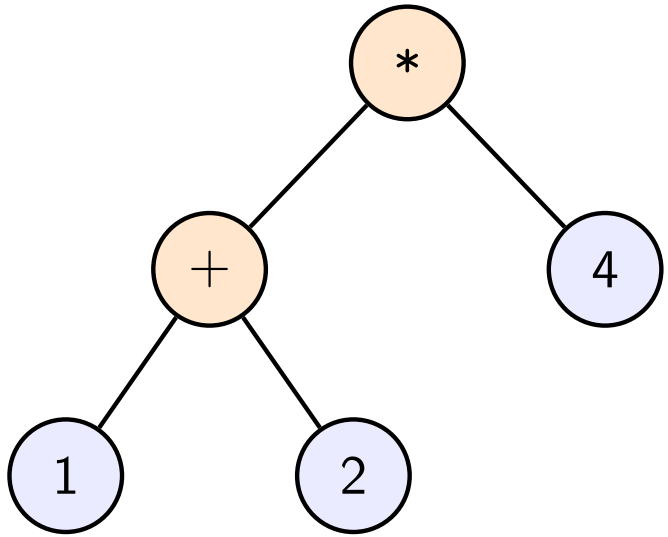
Review 9b: Where Do the Parentheses Go?

For an **operator** node the function calls itself on `root->left`, then on `root->right`.

Q1. Where does the code print `(`?

(A) Before the call on `root->left`
(B) Between the two recursive calls
(C) After the call on `root->right`
(D) Only when `root == nullptr`

Tree for `(1 + 2) * 4`



Review 9b: Where Do the Parentheses Go?

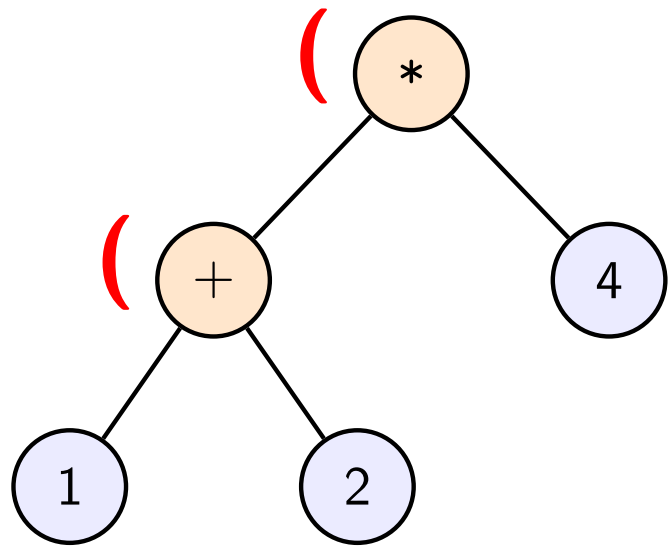
For an **operator** node the function calls itself on `root->left`, then on `root->right`.

Q1. Where does the code print `(`?

(A) **Before the call on `root->left`**
(B) Between the two recursive calls
(C) After the call on `root->right`
(D) Only when `root == nullptr`

Answer (A). Print `(` on entering the operator node, before going left.

Tree for `(1 + 2) * 4`



`(` is printed before going left.

Review 9b: Where Do the Parentheses Go?

For an **operator** node the function calls itself on `root->left`, then on `root->right`.

Q1. Where does the code print `(`?

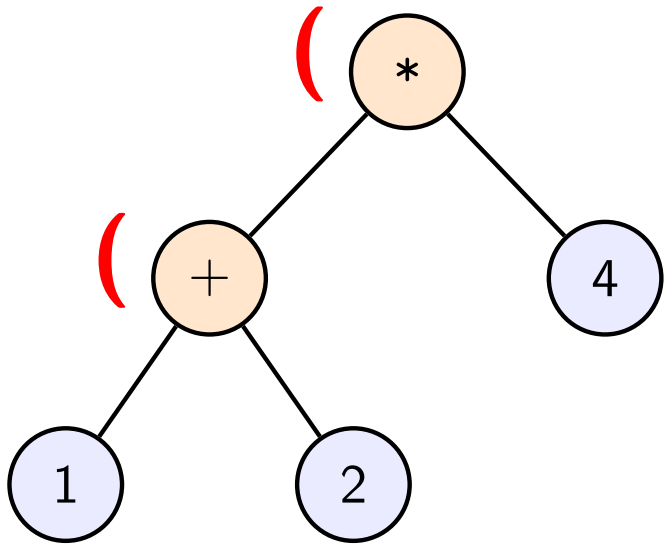
(A) **Before the call on `root->left`**
(B) Between the two recursive calls
(C) After the call on `root->right`
(D) Only when `root == nullptr`

Answer (A). Print `(` on entering the operator node, before going left.

Q2. Where does the code print `)`?

(A) Before any recursive call
(B) Right after printing the operator
(C) After the call on `root->right` returns
(D) Only for number nodes

Tree for `(1 + 2) * 4`



`(` is printed before going left.

Review 9b: Where Do the Parentheses Go?

For an **operator** node the function calls itself on `root->left`, then on `root->right`.

Q1. Where does the code print `(`?

(A) **Before the call on `root->left`**
(B) Between the two recursive calls
(C) After the call on `root->right`
(D) Only when `root == nullptr`

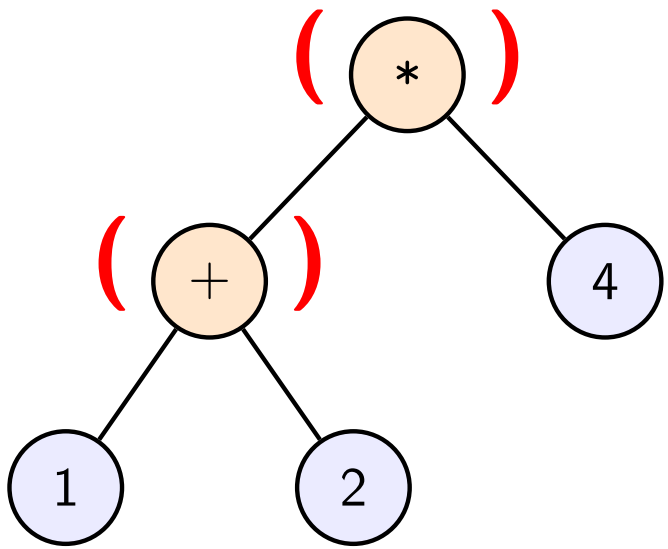
Answer (A). Print `(` on entering the operator node, before going left.

Q2. Where does the code print `)`?

(A) Before any recursive call
(B) Right after printing the operator
(C) **After the call on `root->right` returns**
(D) Only for number nodes

Answer (C). Print `)` after the whole right subtree is done, so the group is closed.

Tree for `(1 + 2) * 4`



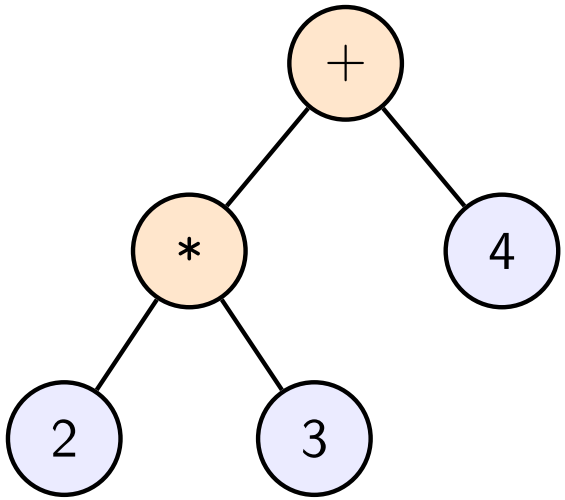
`(` before going left, `)` after the right subtree.

Output: `((1 + 2) * 4)`

Review 9c: Your Turn: Write `printInOrderWParen`

Write the function **on paper**. Use the `Node` definition from slide 15 (Review 7); assume every operator is binary.

Task. `void printInOrderWParen(const Node* root)` prints the tree in-order with parentheses. For the tree on the right it must print `((2 * 3) + 4)`.



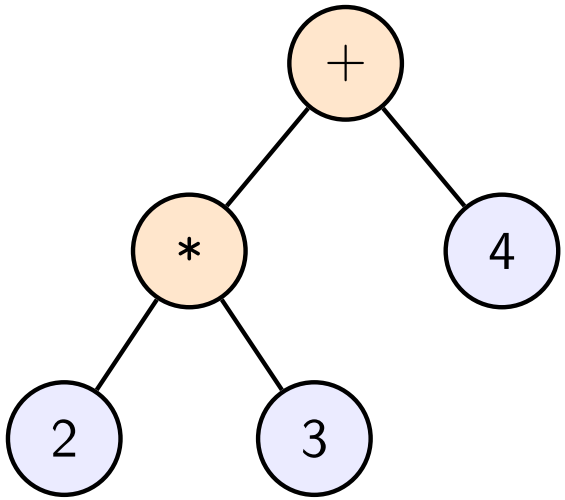
Plan your code in this order:

1. the base case (`root` is `nullptr`);
2. a `bool`: is this node an operator?;
3. print `(` if it is an operator;
4. recursive call on the left subtree;
5. print the token (spaces around an operator);
6. recursive call on the right subtree;
7. print `)` if it is an operator.

Review 9c: Your Turn: Write `printInOrderWParen`

Write the function **on paper**. Use the `Node` definition from slide 15 (Review 7); assume every operator is binary.

Task. `void printInOrderWParen(const Node* root)` prints the tree in-order with parentheses. For the tree on the right it must print `((2 * 3) + 4)`.



Plan your code in this order:

1. the base case (`root` is `nullptr`);
2. a `bool`: is this node an operator?;
3. print `(` if it is an operator;
4. recursive call on the left subtree;
5. print the token (spaces around an operator);
6. recursive call on the right subtree;
7. print `)` if it is an operator.

Check yourself. Trace your code on the tree: `(` twice, then `2 * 3`, `)`, `+ 4`, `)`. The next slide shows the code.

Review 9d: Check Your Code: In-order with Parentheses

Plain `inorder` prints `5 + 1 + 2 * 4 - 3`, which loses the shape. `printInOrderWParen` adds the parentheses back: `((5 + ((1 + 2) * 4)) - 3)`. **Assume** every operator is binary and has two non-null children; number nodes have no children (use the `Node` definition from slide 15, Review 7).

```
void printInOrderWParen(const Node* root) {
    if (root == nullptr) { return; }
    // Binary operators have two children; numbers are leaves.
    bool isOperator = (root->left != nullptr);
    if (isOperator) { std::cout << "("; }
    printInOrderWParen(root->left);
    if (isOperator) { std::cout << " " << root->token << " "; }
    else            { std::cout << root->token; }
    printInOrderWParen(root->right);
    if (isOperator) { std::cout << ")"; }
}
```

Rule. A number prints just itself. An operator prints `(`, its left expression, itself, its right expression, then `)`.

Review 9d: Check Your Code: In-order with Parentheses

Plain `inorder` prints `5 + 1 + 2 * 4 - 3`, which loses the shape. `printInOrderWParen` adds the parentheses back: `((5 + ((1 + 2) * 4)) - 3)`. **Assume** every operator is binary and has two non-null children; number nodes have no children (use the `Node` definition from slide 15, Review 7).

```
void printInOrderWParen(const Node* root) {
    if (root == nullptr) { return; }
    // Binary operators have two children; numbers are leaves.
    bool isOperator = (root->left != nullptr);
    if (isOperator) { std::cout << "("; }
    printInOrderWParen(root->left);
    if (isOperator) { std::cout << " " << root->token << " "; }
    else            { std::cout << root->token; }
    printInOrderWParen(root->right);
    if (isOperator) { std::cout << ")"; }
}
```

Rule. A number prints just itself. An operator prints `(`, its left expression, itself, its right expression, then `)`.

Build bottom up: `(1 + 2)` → `((1 + 2) * 4)` → `(5 + ((1 + 2) * 4))` → `((5 + ((1 + 2) * 4)) - 3)`.

Review 9d: Check Your Code: In-order with Parentheses

Plain `inorder` prints `5 + 1 + 2 * 4 - 3`, which loses the shape. `printInOrderWParen` adds the parentheses back: `((5 + ((1 + 2) * 4)) - 3)`. **Assume** every operator is binary and has two non-null children; number nodes have no children (use the `Node` definition from slide 15, Review 7).

```
void printInOrderWParen(const Node* root) {
    if (root == nullptr) { return; }
    // Binary operators have two children; numbers are leaves.
    bool isOperator = (root->left != nullptr);
    if (isOperator) { std::cout << "("; }
    printInOrderWParen(root->left);
    if (isOperator) { std::cout << " " << root->token << " "; }
    else            { std::cout << root->token; }
    printInOrderWParen(root->right);
    if (isOperator) { std::cout << ")"; }
}
```

Rule. A number prints just itself. An operator prints `(`, its left expression, itself, its right expression, then `)`.

Build bottom up: `(1 + 2)` → `((1 + 2) * 4)` → `(5 + ((1 + 2) * 4))` → `((5 + ((1 + 2) * 4)) - 3)`.

Remember: print `(` on entering an operator node, and `)` after returning from its

Review 10: Where Parentheses Come From

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`. **Predict the output after each step**, then reveal. Rows show output events and resumptions, not separate function calls; calls on `nullptr` are omitted.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	
2	`printInOrderWParen(+)`	operator: print (	
3	`printInOrderWParen(1)`	leaf: print 1	
4	back in +	print +	
5	`printInOrderWParen(2)`	leaf: print 2	
6	back in +	operator: print)	
7	back in *	print *	
8	`printInOrderWParen(4)`	leaf: print 4	
9	back in *	operator: print)	

Review 10: Where Parentheses Come From

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`. **Predict the output after each step**, then reveal. Rows show output events and resumptions, not separate function calls; calls on `nullptr` are omitted.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	(
2	`printInOrderWParen(+)`	operator: print (	
3	`printInOrderWParen(1)`	leaf: print 1	
4	back in +	print +	
5	`printInOrderWParen(2)`	leaf: print 2	
6	back in +	operator: print)	
7	back in *	print *	
8	`printInOrderWParen(4)`	leaf: print 4	
9	back in *	operator: print)	

Review 10: Where Parentheses Come From

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`. **Predict the output after each step**, then reveal. Rows show output events and resumptions, not separate function calls; calls on `nullptr` are omitted.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	(
2	`printInOrderWParen(+)`	operator: print (	((
3	`printInOrderWParen(1)`	leaf: print 1	
4	back in +	print +	
5	`printInOrderWParen(2)`	leaf: print 2	
6	back in +	operator: print)	
7	back in *	print *	
8	`printInOrderWParen(4)`	leaf: print 4	
9	back in *	operator: print)	

Review 10: Where Parentheses Come From

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`. **Predict the output after each step**, then reveal. Rows show output events and resumptions, not separate function calls; calls on `nullptr` are omitted.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	(
2	`printInOrderWParen(+)`	operator: print (	((
3	`printInOrderWParen(1)`	leaf: print 1	((1
4	back in +	print +	
5	`printInOrderWParen(2)`	leaf: print 2	
6	back in +	operator: print)	
7	back in *	print *	
8	`printInOrderWParen(4)`	leaf: print 4	
9	back in *	operator: print)	

Review 10: Where Parentheses Come From

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`. **Predict the output after each step**, then reveal. Rows show output events and resumptions, not separate function calls; calls on `nullptr` are omitted.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	(
2	`printInOrderWParen(+)`	operator: print (	((
3	`printInOrderWParen(1)`	leaf: print 1	((1
4	back in +	print +	((1 +
5	`printInOrderWParen(2)`	leaf: print 2	
6	back in +	operator: print)	
7	back in *	print *	
8	`printInOrderWParen(4)`	leaf: print 4	
9	back in *	operator: print)	

Review 10: Where Parentheses Come From

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`. **Predict the output after each step**, then reveal. Rows show output events and resumptions, not separate function calls; calls on `nullptr` are omitted.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	(
2	`printInOrderWParen(+)`	operator: print (	((
3	`printInOrderWParen(1)`	leaf: print 1	((1
4	back in +	print +	((1 +
5	`printInOrderWParen(2)`	leaf: print 2	((1 + 2
6	back in +	operator: print)	
7	back in *	print *	
8	`printInOrderWParen(4)`	leaf: print 4	
9	back in *	operator: print)	

Review 10: Where Parentheses Come From

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`. **Predict the output after each step**, then reveal. Rows show output events and resumptions, not separate function calls; calls on `nullptr` are omitted.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	`(`
2	`printInOrderWParen(+)`	operator: print (	`((`
3	`printInOrderWParen(1)`	leaf: print 1	`((1`
4	back in +	print +	`((1 +`
5	`printInOrderWParen(2)`	leaf: print 2	`((1 + 2`
6	back in +	operator: print)	`((1 + 2)`
7	back in *	print *	
8	`printInOrderWParen(4)`	leaf: print 4	
9	back in *	operator: print)	

Review 10: Where Parentheses Come From

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`. **Predict the output after each step**, then reveal. Rows show output events and resumptions, not separate function calls; calls on `nullptr` are omitted.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	`(`
2	`printInOrderWParen(+)`	operator: print (	`((`
3	`printInOrderWParen(1)`	leaf: print 1	`((1`
4	back in +	print +	`((1 +`
5	`printInOrderWParen(2)`	leaf: print 2	`((1 + 2`
6	back in +	operator: print)	`((1 + 2)`
7	back in *	print *	`((1 + 2) *`
8	`printInOrderWParen(4)`	leaf: print 4	
9	back in *	operator: print)	

Review 10: Where Parentheses Come From

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`. **Predict the output after each step**, then reveal. Rows show output events and resumptions, not separate function calls; calls on `nullptr` are omitted.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	`(`
2	`printInOrderWParen(+)`	operator: print (	`((`
3	`printInOrderWParen(1)`	leaf: print 1	`((1`
4	back in +	print +	`((1 +`
5	`printInOrderWParen(2)`	leaf: print 2	`((1 + 2`
6	back in +	operator: print)	`((1 + 2)`
7	back in *	print *	`((1 + 2) *`
8	`printInOrderWParen(4)`	leaf: print 4	`((1 + 2) * 4`
9	back in *	operator: print)	

Review 10: Where Parentheses Come From

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`. **Predict the output after each step**, then reveal. Rows show output events and resumptions, not separate function calls; calls on `nullptr` are omitted.

#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	`(`
2	`printInOrderWParen(+)`	operator: print (	`((`
3	`printInOrderWParen(1)`	leaf: print 1	`((1`
4	back in +	print +	`((1 +`
5	`printInOrderWParen(2)`	leaf: print 2	`((1 + 2`
6	back in +	operator: print)	`((1 + 2)`
7	back in *	print *	`((1 + 2) *`
8	`printInOrderWParen(4)`	leaf: print 4	`((1 + 2) * 4`
9	back in *	operator: print)	`((1 + 2) * 4)`

Review 10: Where Parentheses Come From

Tree for $(1 + 2) * 4$: root `*`, left child `+` (children `1`, `2`), right child `4`. **Predict the output after each step**, then reveal. Rows show output events and resumptions, not separate function calls; calls on `nullptr` are omitted.

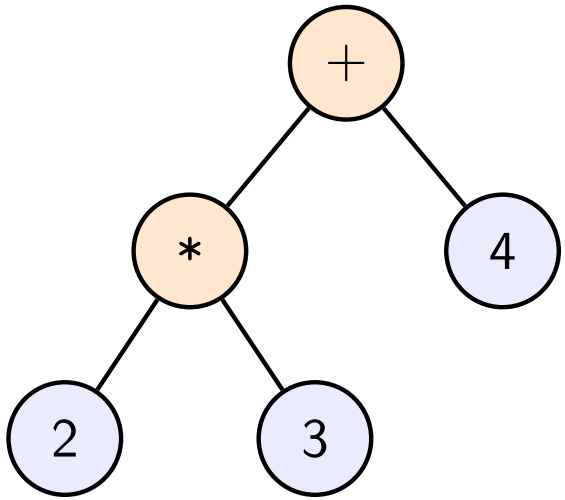
#	Call	Action	Output so far
1	`printInOrderWParen(*)`	operator: print (	`(`
2	`printInOrderWParen(+)`	operator: print (	`((`
3	`printInOrderWParen(1)`	leaf: print 1	`((1`
4	back in +	print +	`((1 +`
5	`printInOrderWParen(2)`	leaf: print 2	`((1 + 2`
6	back in +	operator: print)	`((1 + 2)`
7	back in *	print *	`((1 + 2) *`
8	`printInOrderWParen(4)`	leaf: print 4	`((1 + 2) * 4`
9	back in *	operator: print)	`((1 + 2) * 4)`

Q. Which line prints the) of an operator node? **Answer.** The one *after* `printInOrderWParen(root->right)`, only if the node is an operator: left, operator and right are all printed, so the group can close. The (goes *before* the recursive calls.

Review 11: Your Turn

Tree for `2 * 3 + 4`. Multiplication first, so + is the root.

Q1. Write the pre-order, in-order, and post-order text.

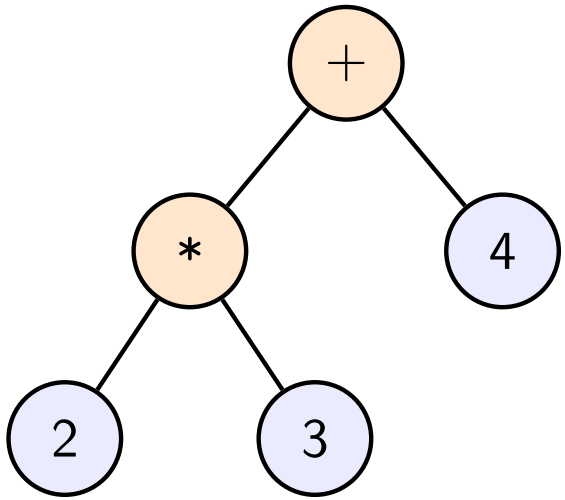


Review 11: Your Turn

Tree for `2 * 3 + 4`. Multiplication first, so + is the root.

Q1. Write the pre-order, in-order, and post-order text.

Pre-order: `+ * 2 3 4`

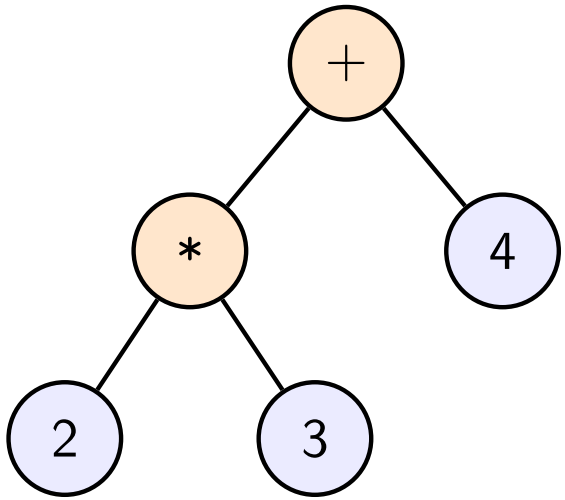


Review 11: Your Turn

Tree for `2 * 3 + 4`. Multiplication first, so + is the root.

Q1. Write the pre-order, in-order, and post-order text.

Pre-order: `+ * 2 3 4`
In-order: `2 * 3 + 4`

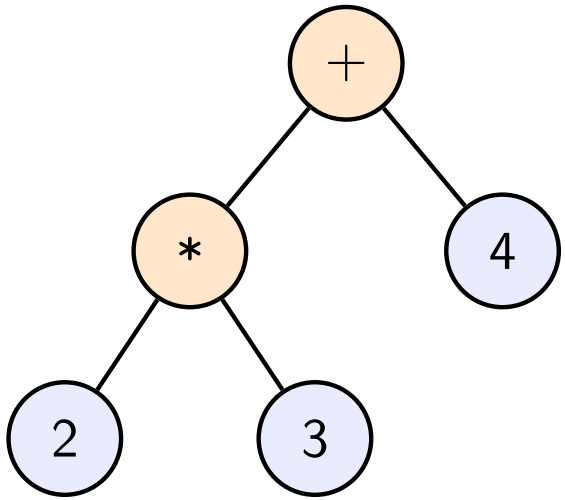


Review 11: Your Turn

Tree for `2 * 3 + 4`. Multiplication first, so + is the root.

Q1. Write the pre-order, in-order, and post-order text.

Pre-order: `+ * 2 3 4`
In-order: `2 * 3 + 4`
Post-order: `2 3 * 4 +`

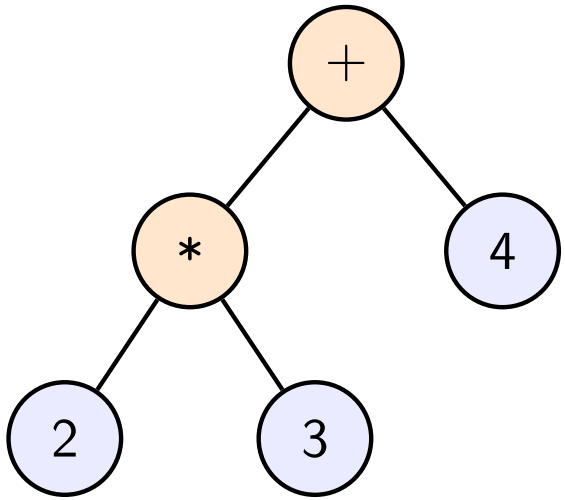


Review 11: Your Turn

Tree for `2 * 3 + 4`. Multiplication first, so + is the root.

Q1. Write the pre-order, in-order, and post-order text.

Pre-order: `+ * 2 3 4`
In-order: `2 * 3 + 4`
Post-order: `2 3 * 4 +`
With parentheses: `((2 * 3) + 4)`



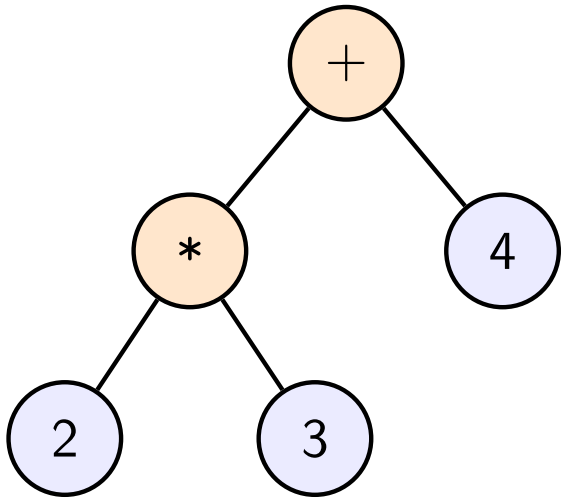
Review 11: Your Turn

Tree for `2 * 3 + 4`. Multiplication first, so + is the root.

Q1. Write the pre-order, in-order, and post-order text.

Pre-order: `+ * 2 3 4`
In-order: `2 * 3 + 4`
Post-order: `2 3 * 4 +`
With parentheses: `((2 * 3) + 4)`

Q2. Which order lists a parent *before* its children? **Pre-order.** Which lists it *after* both? **Post-order.**



Review 12: Reading Backward

Q. The post-order text `5 2 - 3 *` came from an expression tree. What is its root? What is the value?

Review 12: Reading Backward

Q. The post-order text `5 2 - 3 *` came from an expression tree. What is its root? What is the value?

Answer. The *last* token of post-order is the root: $*$. Its left subtree is $5 - 2 = 3$ and its right subtree is 3, so the value is $3 \cdot 3 = \mathbf{9}$ (the infix is $(5 - 2) * 3$, which needs parentheses).

Review 12: Reading Backward

Q. The post-order text `5 2 - 3 *` came from an expression tree. What is its root? What is the value?

Answer. The *last* token of post-order is the root: $*$. Its left subtree is $5 - 2 = 3$ and its right subtree is 3, so the value is $3 \cdot 3 = \mathbf{9}$ (the infix is $(5 - 2) * 3$, which needs parentheses).

Q. Why is the root always last in post-order, and first in pre-order?

Review 12: Reading Backward

Q. The post-order text `5 2 - 3 *` came from an expression tree. What is its root? What is the value?

Answer. The *last* token of post-order is the root: $*$. Its left subtree is $5 - 2 = 3$ and its right subtree is 3, so the value is $3 \cdot 3 = \mathbf{9}$ (the infix is $(5 - 2) * 3$, which needs parentheses).

Q. Why is the root always last in post-order, and first in pre-order?
Answer. Post-order visits both subtrees and then the root; pre-order visits the root before the subtrees. In-order puts the root in the *middle*, which is why its text needs parentheses to show the shape.

Summary and Preview

- **Choose, explore, undo:** when you explore another choice using shared mutable state, restore the state you changed for the previous choice.
- **Pruning:** skip branches that cannot produce a needed new solution. `printUnique` skips repeated choices at the same recursion level.
- **Task B:** the same search as Task A, but the answer carried up is an *expression*.
- **An expression is a tree:** the root is the outermost operation; leaves are numbers; for `+ - * /`, equal precedence associates left to right.
- **Pre-, in-, post-order:** the same three lines in a different order; *where the root is printed* is the only difference.
- **Plain in-order loses parentheses.** `printInOrderWParen` adds them: `(` before the calls, `)` after. Post-order needs none.

Next class

Post-order is the text a stack evaluates directly. We evaluate it, then **convert** infix to postfix (shunting-yard).